DroneArjuna GCS

Connecting a Real Drone over MAVLink

Step-by-step guide for linking your flight controller (firmware) to the DroneArjuna software, based on the DroneArjuna codebase (pymavlink backend, com_bridge, Fleet UI).

Audience	Operators / integrators bringing up a physical vehicle
Firmware	ArduPilot (Copter / Plane) or PX4, speaking MAVLink (v2 recommended)
Links covered	USB cable, SiK telemetry radio, UDP (Wi-Fi / companion computer / MAVProxy), TCP, HF modem
Source of truth	backend/app/modules/drone_control/*, utils/mavlink_utils.py, com_bridge/com_bridge.py, docs/MAVLink-Communication.md

1. How DroneArjuna talks to a drone

DroneArjuna acts as a MAVLink **ground control station** (system id 255, component type GCS). The FastAPI backend (running in Docker) opens a pymavlink connection per drone, waits for the vehicle's first HEARTBEAT, requests telemetry streams, then runs a read loop that feeds telemetry to the UI and sends your commands.

```
Flight controller (ArduPilot / PX4)
       |  MAVLink over: USB | SiK radio | UDP | TCP | HF modem
       v
[Windows host]  com_bridge.py  (serial COM port -> TCP 5762, discovery HTTP 5761)
       |   (only for serial/USB links on Windows + Docker)
       v
[Docker] backend  MAVLinkManager -> read loop -> TelemetryProcessor -> StateManager -> UI
                                  <- CommandController (validate, encode, send, wait COMMAND_ACK)
```

Why com_bridge exists: Docker containers on Windows cannot open Windows COM ports. com_bridge runs on the Windows host, forwards the serial device to TCP, and the backend reaches it at `host.docker.internal:5762`. Network links (UDP/TCP) do not need it.

Supported transports (what the API accepts)

Transport	pymavlink string built	Typical use
udp	udpin:0.0.0.0:<port>	Drone/companion/MAVProxy sends to this PC (default 14550). Host field is ignored; backend always listens on all interfaces.
tcp	tcp:<host>:<port>	com_bridge (host.docker.internal:5762), ArduPilot SITL (5760/5762).
serial	<device>,<baud>	Serial device visible *inside the container* (Linux host / usbipd). On Windows use the bridge instead.
hf_serial / hf_tcp	serial / tcp	HF radio modem; adds message throttling and longer timeouts (Section 9).

NOTE: The REST schema (ConnectRequest) accepts only udp, tcp, serial, hf_serial and hf_tcp. The udp_out and sitl transports exist in mavlink_utils.build_connection_string but are not selectable from the API/UI.

2. Prerequisites

2.1 Vehicle / firmware side

- ArduPilot Copter/Plane or PX4 firmware that outputs MAVLink HEARTBEAT. The decoder has mode tables for ArduCopter, ArduPlane and PX4.
- A unique MAVLink system id per vehicle (ArduPilot parameter `SYSID_THISMAV`). Register the same value in DroneArjuna's drone record (Section 4).
- A telemetry port configured for MAVLink (Section 3). On ArduPilot: `SERIALn_PROTOCOL` = 1 (MAVLink1) or 2 (MAVLink2), and `SERIALn_BAUD` to match the link (57 = 57600, 115 = 115200).
- Working GPS (3D fix, at least 6 satellites) and battery monitor if you intend to arm from the software - the command validator refuses otherwise.
- For safety: configure the autopilot's own failsafes (GCS-loss, battery, geofence). DroneArjuna drops a silent link after 10 s (45 s on HF) but the vehicle must protect itself.

2.2 Software / host side

- Docker Desktop with the DroneArjuna stack (`.env` copied from `.env.example`, secrets filled in).
- Python on the Windows host with `pyserial` (for com_bridge). Close Mission Planner / QGroundControl / any app holding the COM port.

- A user account with role **FLIGHT_CONTROLLER** or higher (connect, autoconnect, commands). VIEWER can only list ports and read telemetry.
- Ports in use: 8000 (API), 3000 (UI), 14550/udp (MAVLink in), 14560/udp (simulator broadcast out), 5761 and 5762 (com_bridge, on the host).

3. Step 1 - Wire / configure the link

Option A - USB cable to a Pixhawk-class board (simplest)

- Plug the USB cable in. The board appears as a COM port on Windows.
- USB-direct links run at 115200 baud regardless of the SERIALn_BAUD parameter (the code probes 115200, 57600, 921600 for USB serial).
- Use com_bridge (Section 5) so the Dockerised backend can read it.

Option B - SiK telemetry radio (433/915 MHz)

- Air-side radio on the autopilot's TELEM port: `SERIAL1_PROTOCOL=2`, `SERIAL1_BAUD=57`.
- Ground-side radio plugged into the PC; shows up as a COM port. Default air baud is 57600, so start com_bridge with baud 57600.
- Use com_bridge as in Option A.

Option C - UDP over Wi-Fi / LTE / companion computer / MAVProxy

- The vehicle side must **send MAVLink to this PC's IP on UDP 14550**. Example with MAVProxy on a companion computer: `mavproxy.py --master=/dev/ttyAMA0,921600 --out=udp:<GCS_PC_IP>:14550`.
- The backend listens (udpin) so the drone side must transmit first; nothing is sent until a datagram arrives.
- **Docker exposure:** docker-compose publishes 14550/udp bound to `${RADIO_BIND_ADDR:-127.0.0.1}`. The default 127.0.0.1 only accepts same-machine traffic. For a remote drone, set `RADIO_BIND_ADDR` in .env to the PC's radio/LAN interface IP and recreate the stack. Open the Windows firewall for UDP 14550 on that interface only.

Option D - TCP

- Point the transport at the TCP endpoint of the vehicle or its bridge (e.g. `tcp:<host>:5760`). ArduPilot SITL also uses this.

Option E - HF modem

- Use transport hf_serial (modem as serial port) or hf_tcp (modem ALE interface over TCP) and choose the modem type (harris / codan / barrett / generic). See Section 9.

4. Step 2 - Register the drone in DroneArjuna

- Log in at `http://localhost:3000` with a FLIGHT_CONTROLLER (or higher) account.
- Open **Settings -> Drone Instance Manager** (Drone Master) and add a drone: **Call Sign***, **Drone Type***, **Serial Number***, **MAVLink ID** (default 1 - set to your SYSID_THISMAV) and optionally home vessel / payload.
- The drone must be active and not in *maintenance* status, otherwise the auto-connector ignores it.
- The connect call is keyed by this record (`drone_instance_id`); telemetry and commands are then tracked under it.

NOTE: Only one vehicle per link. After the first HEARTBEAT the backend binds to that system/component (target_system / target_component). If two vehicles share one UDP port, give each its own port and its own drone record.

5. Step 3 - Start the bridge (serial / USB / SiK on Windows)

Skip this step for pure UDP/TCP links. From the DroneArjuna folder on the Windows host:

```
# Full stack + bridge in one go (recommended)
.\start.ps1                          # auto-pick USB serial, 115200 baud
.\start.ps1 -ComPort COM4 -Baud 57600   # SiK radio on COM4
.\start.ps1 -Build                   # force Docker image rebuild

# Bridge only
python com_bridge\com_bridge.py COM4 57600 5762
#   args: [COM_PORT|auto] [BAUD] [TCP_PORT]   defaults: auto, 115200, 5762
```

Item	Detail
Discovery endpoint	http://localhost:5761/ports returns {connected, active_port, baud, tcp_port, ports[]}
MAVLink TCP port	5762 by default; falls back to 5764, 5770, 5760 or an OS-chosen port if busy (reported in discovery)
Single client	Only one MAVLink client at a time; a second is rejected. Close other GCS software.
Single instance	bridge.lock prevents two bridges fighting over a COM port
Security	BRIDGE_BIND_ADDR (default 0.0.0.0) and BRIDGE_ALLOWED_CLIENTS (default loopback + private ranges) restrict who may connect. The MAVLink stream itself has no authentication - tighten these for field use.
Auto-start	install-autostart.ps1 / install-bridge-service.ps1 register the bridge to start with Windows

Verify: open `http://localhost:5761/ports` - you should see `"connected": true` and your COM port as `active_port`.

6. Step 4 - Connect from the software

6.1 Auto-connect (zero configuration)

- The backend runs a background **auto-connector**. Every 3 s it polls the bridge; when a cable is plugged in it immediately probes for a heartbeat. It also retries unconnected active drones every 15 s.
- It tries candidates concurrently: the bridge TCP port first, then container serial ports (115200/57600/921600 for USB, 57600/115200 otherwise), then fallbacks tcp:5762, tcp:5760, udp:14550/14551/14560. First heartbeat wins (8 s probe timeout). Failing fallbacks go into a 120 s cooldown after two failures.
- Typical time from plug-in to connected: about 7 s.
- Manual trigger: Fleet workspace connect modal, or `POST /api/drone-control/autoconnect` with `{"drone_instance_id": N}` (probes sequentially, 6 s each; returns 503 if nothing answers, 409 if already connected).

6.2 Manual connect from the Fleet UI

- Fleet workspace -> select the drone -> **Connect**. Pick a preset or fill the fields:

Preset in ConnectModal	Transport	Parameters
SITL UDP 14550	udp	host 0.0.0.0, port 14550
Windows COM Bridge TCP 5762	tcp	host.docker.internal, port 5762
Pixhawk USB Direct 115200	serial	/dev/ttyUSB0, 115200 (Linux/container only)
SiK Telemetry Radio 57600	serial	/dev/ttyUSB0, 57600 (Linux/container only)
HF Harris Serial	hf_serial	/dev/ttyUSB0, 9600, modem harris

On Windows with a USB/SiK link, choose the 'Windows COM Bridge TCP 5762' preset - the serial presets refer to devices inside the container.

6.3 Connect through the REST API

```
# list ports (VIEWER+): bridge COM ports, container serial devices, network endpoints
GET  /api/drone-control/ports

# connect (FLIGHT_CONTROLLER+) - USB/SiK via the bridge
POST /api/drone-control/connect
{ "drone_instance_id": 3, "transport": "tcp",
  "host": "host.docker.internal", "port": 5762 }

# connect - UDP
{ "drone_instance_id": 3, "transport": "udp", "port": 14550 }

# connect - serial inside container
{ "drone_instance_id": 3, "transport": "serial",
  "serial_port": "/dev/ttyUSB0", "baud_rate": 57600 }

POST /api/drone-control/disconnect/{drone_id}
GET  /api/drone-control/status              # connected drones + live state
```

Success returns `201 {"detail":"Connected"}`; failure returns `503` ("Connection failed or heartbeat timed out"). All calls need the bearer token from `/api/auth` login.

7. What happens during connect (and what you should see)

#	Backend action	Visible result
1	Opens link with source_system=255, applies MAVLink 2 signing if configured	-
2	Waits for first HEARTBEAT (10 s UDP/TCP/serial; 45 s HF)	Timeout -> 503 / 'heartbeat timeout' in logs
3	Requests streams: REQUEST_DATA_STREAM plus MAV_CMD_SET_MESSAGE_INTERVAL for GPS_RAW_INT 2 Hz, RC_CHANNELS 2 Hz, GLOBAL_POSITION_INT 4 Hz, ATTITUDE 10 Hz, VFR_HUD 4 Hz, SYS_STATUS 2 Hz	Telemetry starts flowing
4	Creates CommandController and read loop; sends GCS heartbeat at 1 Hz	Drone shows connected in Fleet
5	Pushes the UI home point (home waypoint of newest mission) via MAV_CMD_DO_SET_HOME; then every 5 s updates home from vessel position (for ship-launched RTL)	RTL target matches the map
6	Marks the drone as used / updates last_seen	Status updated in Drone Master

Messages the backend consumes: HEARTBEAT, GLOBAL_POSITION_INT, ATTITUDE, VFR_HUD, SYS_STATUS, GPS_RAW_INT, RC_CHANNELS, BATTERY_STATUS, COMMAND_ACK.

Live data: `GET /api/drone-control/telemetry/{id}`, `/telemetry/{id}/gauges`, `/history`, `/battery-log`, plus the WebSocket telemetry stream used by the UI.

Link supervision: the connection is dropped automatically after 10 s of silence (45 s on HF) or 10 consecutive read errors, and the auto-connector will then retry.

8. Step 5 - Fly: commands and safety checks

Commands go through `POST /api/drone-control/command` (`{"drone_id":N,"command":"arm","params":{}}`, FLIGHT_CONTROLLER+, audit-logged).

Command	MAVLink sent	Software pre-check
arm / disarm	COMMAND_LONG MAV_CMD_COMPONENT_ARM_DISARM	arm denied if battery < 20% or no 3D fix with at least 6 satellites
takeoff	COMMAND_LONG MAV_CMD_NAV_TAKEOFF	must be armed; altitude 1-500 m
set_mode	SET_MODE	mode must exist in the autopilot's mode_mapping()
rtl	SET_MODE RTL or SMART_RTL	-

Command	MAVLink sent	Software pre-check
land	SET_MODE LAND	-
goto	MISSION_ITEM_INT (guided goto)	-
velocity	SET_POSITION_TARGET_LOCAL_NED	-
emergency_stop	MAV_CMD_DO_FLIGHTTERMINATION	-

- ACK handling: COMMAND_ACK is awaited up to 5 s with 2 retries (8 s on HF). set_mode/rtl/land/goto/velocity have no ACK and resolve after a short grace period; confirm mode changes from the HEARTBEAT shown in the UI.
- Only one arm/disarm may be pending at once (they share a command id).
- A drawn geofence breach triggers an automatic RTL through the geofence consumer, unless airspace enforcement is disabled for the mission.
- Last 200 commands per drone are retained for audit.

SAFETY: First flights: bench-test with propellers removed, verify telemetry, mode changes and arm/disarm, then test RTL/land in a safe open area with a manual RC override available. Do not rely on the GCS link as the only failsafe.

9. Optional: HF ship-to-shore datalink

- Choose hf_serial or hf_tcp and set the modem type. The same MAVLink stream is carried, but an adapter throttles it.
- **Priority filter:** COMMAND_ACK never limited; position/battery/attitude/HEARTBEAT at most 2 Hz; VFR_HUD and mission progress 0.5 Hz; STATUSTEXT, PARAM_VALUE, raw IMU and RC channels blocked.
- **Timeouts:** heartbeat 45 s, ACK 8 s. State machine: CONNECTED -> DEGRADED after 20 s silence -> LOST after 120 s (failsafe applies).
- Expect 1.2-9.6 kbps; start streams conservatively and avoid parameter downloads over HF.

10. Optional: MAVLink 2 message signing

- Generate a key: `python -c "import secrets;print(secrets.token_hex(32))"` and set `MAVLINK_SIGNING_KEY=<64 hex chars>` in .env (startup validates length). Empty means signing off.
- The **same key must be loaded in the autopilot** (e.g. via Mission Planner / MAVProxy 'setup signing'). With signing on, the backend signs outgoing frames and discards unsigned/forged incoming frames - only RADIO_STATUS is exempt.
- If you enable the key here but not on the vehicle, you will see no telemetry (heartbeat timeout). Enable signing last, after the link is proven unsigned.
- Restart the backend after changing the key. It is applied at connection open.

11. Troubleshooting

Symptom	Likely cause	Fix
503 'heartbeat timed out' on UDP	Nothing sending to PC:14550, or Docker only bound to 127.0.0.1	Point MAVProxy/autopilot output to PC IP:14550; set RADIO_BIND_ADDR to the LAN IP; allow firewall UDP 14550; recreate stack
Bridge shows connected:false	No COM port found / port busy	Check Device Manager; close Mission Planner/QGC; run com_bridge with explicit COM port; check bridge.err.log
Bridge: 'another MAVLink client is already connected'	Second client (or stale backend probe) on 5762	Disconnect the first; only one client allowed

Symptom	Likely cause	Fix
Heartbeat OK but no GPS/RSSI values	Autopilot ignoring REQUEST_DATA_STREAM	Backend also sends SET_MESSAGE_INTERVAL; check SRn_* params and firmware version
Connects then drops after ~10 s	Link silent (radio range, wrong baud) or only one-way link	Match baud on both radios; check antennas; verify the vehicle receives the GCS heartbeat
No data after setting signing key	Key mismatch / vehicle unsigned	Set same key on autopilot or clear MAVLINK_SIGNING_KEY
Arm command DENIED	Battery < 20% or GPS not 3D fix / fewer than 6 sats	Wait for GPS fix; charge battery; clear autopilot pre-arm errors (STATUSTEXT in GCS)
Drone not auto-connecting	Instance inactive, in maintenance, or none created	Create/activate it in Drone Master
409 already connected	Connection exists	POST /disconnect/{id} then reconnect
Wrong vehicle / garbage data	Two vehicles on one link or same SYSID_THISMAV	One vehicle per link; unique system ids

12. Quick checklist

- [ ] Firmware outputs MAVLink on the chosen port; SYSID_THISMAV unique; GPS and battery healthy
- [ ] .env filled in; stack running (start.ps1 or docker compose up)
- [ ] com_bridge running and http://localhost:5761/ports shows connected (serial/USB/SiK only)
- [ ] RADIO_BIND_ADDR / firewall set for remote UDP links
- [ ] Drone instance created, active, MAVLink ID matches SYSID_THISMAV
- [ ] Logged in as FLIGHT_CONTROLLER or higher
- [ ] Auto-connect or manual connect succeeds; telemetry updating in Fleet/Fly
- [ ] Optional: signing key matched on both sides
- [ ] Autopilot failsafes configured; bench test with props off before first flight

Related docs in the repository: docs/MAVLink-Communication.md (internal design), docs/TLS.md, README.MD (stack start-up). Tests: test_auto_connector.py, test_command_controller.py, test_mavlink_signing.py, test_mavlink_broadcaster.py.